

Desarrollo de un dispositivo de comunicación de equipos de monitoreo y control energético para la domótica de motorhomes

Esp. Ing. Matías Nahuel Rodriguez

Maestría en Internet de las Cosas

Director: Mg. Lic. Leopoldo Alfredo Zimperz (FIUBA)

Jurados:

Mg. Ing. Jonathan Cagua (FIUBA)
Dr. Lic. Roberto Federico Farfán (FIUBA)
Mg. Ing. Raúl Emilio Romero (FIUBA)

Ciudad de Rosario, Santa Fe, junio de 2026

Resumen

El presente trabajo consiste en el desarrollo de un dispositivo electrónico destinado a integrar equipos del ecosistema Victron Energy a la red de domótica de motorhomes desarrollada por la empresa Enertik Argentina. Su implementación permite ampliar las capacidades de la red de domótica existente y posibilita a los usuarios el monitoreo y control de distintos sistemas del vehículo a través de dispositivos móviles.

Para su desarrollo fue necesario aplicar conocimientos que abarcan la implementación de firmware para microcontroladores, aplicaciones orientadas a Internet de las Cosas destinadas a sistemas Android y el desarrollo de placas de circuitos impresos para productos comerciales.

Índice general

Resumen	I
1. Introducción general	1
1.1. Domótica en motorhomes	1
1.2. Motivaciones del trabajo	1
1.3. Estado del arte	2
1.4. Objetivos y alcances	4
2. Introducción específica	7
2.1. Arquitectura general del sistema	7
2.2. Protocolos de comunicación	8
2.2.1. Protocolo CAN	8
2.2.2. Protocolo Modbus TCP	8
2.2.3. Solicitudes HTTP	9
2.3. Componentes de hardware	9
2.3.1. Microcontrolador ESP32	9
2.3.2. Módulo transceptor	10
2.3.3. Módulo Ethernet W5100	11
2.4. Herramientas de software	11
2.4.1. Entorno de desarrollo Arduino	11
2.4.2. Funcionamiento del ESP32	12
2.4.3. Desarrollo de la aplicación móvil	12
3. Diseño e implementación	15
3.1. Desarrollo de hardware	15
3.1.1. Comunicación con la red Enertik	16
3.1.2. Comunicación con la red Victron	18
3.1.3. Desarrollo de la placa final y listado de componentes	19
3.2. Desarrollo de firmware	20
3.2.1. Tarea: recepción UART	21
3.2.2. Tarea: decodificación de comando CAN	21
3.2.3. Tarea: comunicación Modbus TCP	22
3.2.4. Tarea: pedido de datos	23
3.2.5. Tarea: procesamiento de comandos recibidos	24
3.2.6. Tarea: control de servidor	24
3.3. Desarrollo de la aplicación	25
3.3.1. Arquitectura funcional de la aplicación	26
3.3.2. Secuencia de interacción entre la aplicación y el servidor	27
3.3.3. Generación de instalador para Android	28
4. Ensayos y resultados	31
4.1. Diseño final de la aplicación	31
4.2. Ensayos sobre la aplicación	33

4.3. Ensayos sobre el sistema	35
4.4. Ensayos de laboratorio	36
4.4.1. Prototipo experimental	37
4.4.2. Ensayos de integración del sistema completo	38
5. Conclusiones	41
5.1. Conclusiones generales	41
5.2. Próximos pasos	42
Bibliografía	43

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control.	3
2.1. Arquitectura general del sistema.	7
2.2. Microcontrolador ESP32 ¹	10
2.3. Funcionamiento del módulo transceptor.	10
2.4. Módulo W5100 ²	11
2.5. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.	13
3.1. Esquema de interconexión entre el ESP32 y los módulos externos.	15
3.2. Módulo transceptor comercial perteneciente a Enertik.	16
3.3. Diagrama de cómo se establece la comunicación CAN entre la red del motorhome y el ESP32.	17
3.4. Diseño de divisor resistivo para adaptación UART.	17
3.5. Diseño final para PCB del equipo.	20
3.6. Relación entre las distintas tareas del firmware y su comunicación con el exterior.	21
3.7. Contenedores dentro de una pantalla.	26
3.8. Arquitectura funcional de la aplicación.	27
3.9. Secuencia de solicitudes GET entre la aplicación y el microcontrolador.	28
3.10. Secuencia de solicitudes POST entre la aplicación y el microcontrolador.	28
4.1. Pestaña home de la aplicación.	31
4.2. Pestaña luces de la aplicación.	32
4.3. Pestaña motores de la aplicación.	32
4.4. Pestaña energía de la aplicación.	33
4.5. Entorno de ejecución y depuración de la aplicación.	33
4.6. Consola del navegador utilizada como depurador.	34
4.7. Archivo JSON recibido en la pestaña home.	35
4.8. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.	36
4.9. Respuesta del servidor ante una solicitud de control.	36
4.10. Prototipo experimental del equipo.	37
4.11. Módulo PRC10 utilizado en los ensayos.	38
4.12. Equipos Enertik utilizados en los ensayos.	39
4.13. Equipo Victron MultiPlus-II y módulo Cerbo GX.	40

Índice de tablas

Capítulo 1

Introducción general

En este capítulo se describe la red de domótica implementada por Enertik, junto con el estado del arte actual, las motivaciones y los objetivos planteados durante la planificación del trabajo.

1.1. Domótica en motorhomes

Este trabajo fue desarrollado para la empresa Enertik Argentina [1]. Esta empresa se dedica principalmente a la comercialización de equipos asociados al campo de las energías renovables. A su vez, dispone de un área de investigación y desarrollo que se especializa en la fabricación y comercialización de equipos destinados a la domótica de motorhomes.

La domótica consiste en la incorporación de sistemas electrónicos que permiten automatizar, monitorear y controlar distintos parámetros asociados a una unidad habitacional. Estos sistemas buscan mejorar el confort del usuario, optimizar el uso de la energía y simplificar la operación de los diferentes dispositivos presentes en el entorno.

En el caso particular de los motorhomes, la domótica se aplica al control de diferentes subsistemas del vehículo, tales como la iluminación, los sistemas de audio, el monitoreo del estado de las baterías, el accionamiento de los motores que operan las partes móviles del vehículo y la supervisión de distintos parámetros del sistema energético. La integración de estos elementos permite centralizar la información y el control de los dispositivos, y así facilitar su operación por parte del usuario.

En este contexto, Enertik desarrolla una red de domótica propia destinada a la integración y control de los diferentes dispositivos presentes en un motorhome. Esta red permite interconectar diversos equipos electrónicos y posibilita su supervisión y control mediante una pantalla que permite visualizar y operar los distintos dispositivos conectados a la red de domótica.

1.2. Motivaciones del trabajo

Fueron varias las motivaciones que impulsaron el desarrollo de este trabajo. En primer lugar, dentro de los distintos dispositivos que forman parte de la red de domótica del motorhome, existe un subsistema encargado de monitorear y gestionar la parte energética del vehículo.

Este sistema se compone principalmente de un conjunto de equipos conformado por una batería, un inversor y un cargador.

La batería constituye el dispositivo de almacenamiento de energía del vehículo. A partir de ella se alimentan los diferentes equipos eléctricos presentes en la unidad.

Por su parte, el inversor es el encargado de transformar la tensión continua proveniente de la batería en tensión alterna. Esto permite alimentar dispositivos eléctricos que normalmente funcionan conectados a redes eléctricas domiciliarias.

Finalmente, el cargador tiene la función de convertir la tensión alterna proveniente de la red eléctrica en una forma adecuada para recargar las baterías y almacenar energía en ellas.

En la actualidad se observa una tendencia en la que los fabricantes de motorhomes reemplazan este conjunto de batería, cargador e inversor por equipos provistos por la marca Victron Energy [2].

Esta tendencia fue la principal impulsora en el desarrollo de este trabajo, ya que fue necesario diseñar un dispositivo que permita acoplar los equipos provistos por la marca Victron a la red de domótica implementada por Enertik.

Asimismo, en los sistemas actuales la pantalla HMI (*Human Machine Interface*) constituye el principal medio de interacción entre el usuario y los dispositivos del vehículo. Si bien su funcionamiento es, en general, confiable, se han registrado situaciones puntuales en las que fallas en dicha interfaz impiden el acceso a las funcionalidades de la domótica.

En estos casos, la imposibilidad de interactuar con el sistema compromete la operación de los distintos dispositivos del vehículo, lo que evidencia la necesidad de disponer de mecanismos alternativos de control.

En este contexto, surge la necesidad de contar con una solución que permita supervisar y controlar de manera unificada los equipos del vehículo, tanto de la red Enertik como de la red Victron, independientemente del medio de acceso disponible.

De esta manera, se abordaron y resolvieron dos problemáticas presentes: por un lado, la migración hacia nuevas tecnologías energéticas por parte de los fabricantes de motorhomes y, por otro, la necesidad de brindar a los usuarios nuevas herramientas para la supervisión y el control del vehículo.

1.3. Estado del arte

En la actualidad, Enertik dispone de una estructura bien definida a la hora de domotizar un motorhome. Entre los distintos dispositivos que componen esta red, se destaca uno en particular denominado PRC10 [3].

La figura 1.1 muestra un esquema básico de cómo se compone la red de domótica en la actualidad.

Como se puede apreciar, la arquitectura se basa en un esquema centralizado, donde el módulo PRC10 actúa como unidad de control principal del sistema. Los diferentes dispositivos presentes en el vehículo se comunican con este módulo, que se encarga de procesar la información recibida y ejecutar las acciones correspondientes sobre los distintos subsistemas.

También, se observa la presencia de una pantalla táctil que actúa como interfaz de usuario entre el sistema de domótica y los ocupantes del vehículo. A través de esta interfaz es posible visualizar diferentes parámetros obtenidos a partir de los sensores distribuidos en la unidad, tales como valores asociados al sistema energético, estados de funcionamiento de los dispositivos conectados y otras variables de interés para el usuario.

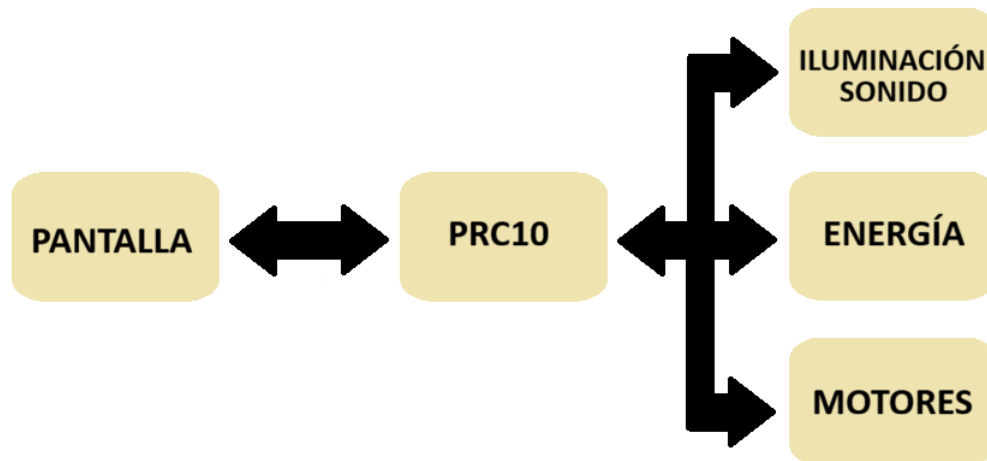


FIGURA 1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control.

Además de permitir la supervisión del estado general del sistema, la pantalla brinda la posibilidad de interactuar con los distintos periféricos presentes en el motorhome. De esta manera, el usuario puede realizar acciones de control sobre diversos dispositivos, como la iluminación, los mecanismos motorizados y otros subsistemas integrados dentro de la red de domótica.

Entre los distintos dispositivos que componen la red de domótica de Enertik, se destacan los siguientes:

- PRC10: módulo central de procesamiento, responsable de la recepción de información proveniente de los demás dispositivos y de la toma de decisiones sobre el funcionamiento del sistema. Además, permite el comando de distintos periféricos, tales como la iluminación interna del vehículo, la apertura y cierre de esclusas de agua, la activación de la bomba de agua y el accionamiento de los motores asociados a las partes móviles del vehículo, como la mesa, la cama levadiza y el toldo.
- Pantalla HMI: interfaz de usuario de tipo táctil que ejecuta el software principal del sistema. Permite la visualización del estado de las variables y la operación de los distintos dispositivos conectados a la red.
- Módulo transceptor: dispositivo que permite la adaptación de la pantalla HMI a la red de comunicación del motorhome.
- Módulo sensor de gases: dispositivo encargado de la medición de la composición de gases en el ambiente. Genera alertas ante la detección de condiciones perjudiciales para la salud.
- Módulo RGB: módulo de potencia destinado al control de la iluminación RGB distribuida en el vehículo.

- Módulo de control de patas: dispositivo responsable del accionamiento del sistema de estabilización del vehículo mediante el control de las patas de apoyo.
- Módulo de control de escalón: dispositivo responsable de la operación del mecanismo del escalón de acceso al vehículo.
- Módulo de relés para luces vehiculares: dispositivo encargado de la adaptación de niveles de tensión para el control de las luces del vehículo, tales como luces delanteras, traseras y señalización asociada al freno de mano.
- Módulo medidor de tanques de agua: módulo encargado de medir y reportar el nivel de llenado de los tanques de agua limpia, gris y negra.

Si bien la arquitectura actual de la red de domótica implementada por Enertik permite una gestión centralizada y eficiente de los distintos dispositivos del motorhome, presenta ciertas limitaciones en términos de flexibilidad y robustez del sistema.

Por un lado, la interacción del usuario depende principalmente de la pantalla HMI, lo que introduce un único punto de acceso al sistema. Si bien el vehículo dispone de controles alternativos de tipo analógico para la operación de algunos subsistemas, estos presentan funcionalidades limitadas y requieren un mayor espacio físico dentro de la unidad.

Por otro lado, en el contexto actual, la incorporación de equipos de la marca Victron no se encuentra integrada dentro de la red de domótica. En estos casos, los dispositivos de gestión energética operan de manera independiente, sin una supervisión unificada junto al resto de los sistemas del vehículo.

En este sentido, se observa la ausencia de mecanismos que permitan integrar estos equipos dentro de la red existente, así como también la falta de alternativas de acceso que complementen o reemplacen la interfaz principal. Estas limitaciones evidencian la necesidad de mejorar la integración y la accesibilidad del sistema de domótica.

1.4. Objetivos y alcances

El objetivo general del presente trabajo es desarrollar una solución que permita integrar sistemas de gestión energética basados en tecnología Victron dentro de la red de domótica de Enertik, así como proporcionar un mecanismo alternativo de acceso para la supervisión y el control del sistema.

A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Diseñar un dispositivo de comunicación capaz de vincular la red de domótica de Enertik con el ecosistema de equipos Victron.
- Implementar los mecanismos necesarios para el intercambio de información entre ambos sistemas, que permitan la supervisión y el control de variables energéticas.
- Desarrollar una aplicación móvil que permita al usuario visualizar el estado del sistema y operar los distintos dispositivos del vehículo.

- Proveer una alternativa de acceso al sistema de domótica que complemente la interfaz principal basada en la pantalla HMI.

En cuanto al alcance del trabajo, se desarrolló un prototipo funcional basado en el microcontrolador ESP32 [4], capaz de actuar como intermediario entre la red de domótica de Enertik y el sistema Victron.

Dentro del ecosistema Victron Energy existe un dispositivo denominado Cerbo GX [5]. Este dispositivo actúa como módulo de control principal de la red de equipos Victron.

El sistema implementado permite la lectura de variables relevantes de dicho módulo y el envío de comandos básicos para el control del equipo. Esta información se integra dentro de la lógica de la red de domótica existente.

Asimismo, se desarrolló una aplicación móvil que permite al usuario visualizar información del sistema y ejecutar acciones de control sobre los dispositivos conectados.

Capítulo 2

Introducción específica

En este capítulo se describen los distintos protocolos de comunicación utilizados en el desarrollo del trabajo, los componentes de hardware empleados para cumplir con los objetivos planteados y las herramientas requeridas para el desarrollo del software.

2.1. Arquitectura general del sistema

La figura 2.1 presenta un esquema general del sistema desarrollado, en el que se ilustran los distintos subsistemas involucrados y sus interconexiones.

Como se puede observar, el dispositivo basado en ESP32 actúa como nodo central del sistema y establece comunicación con la red de domótica de Enertik mediante CAN (*Controller Area Network*) [6], con el ecosistema Victron a través de Modbus TCP sobre Ethernet [7], y con la aplicación móvil mediante solicitudes HTTP (*HyperText Transfer Protocol*) [8] sobre Wi-Fi.

Esta arquitectura permite integrar subsistemas previamente independientes dentro de una única plataforma de control.

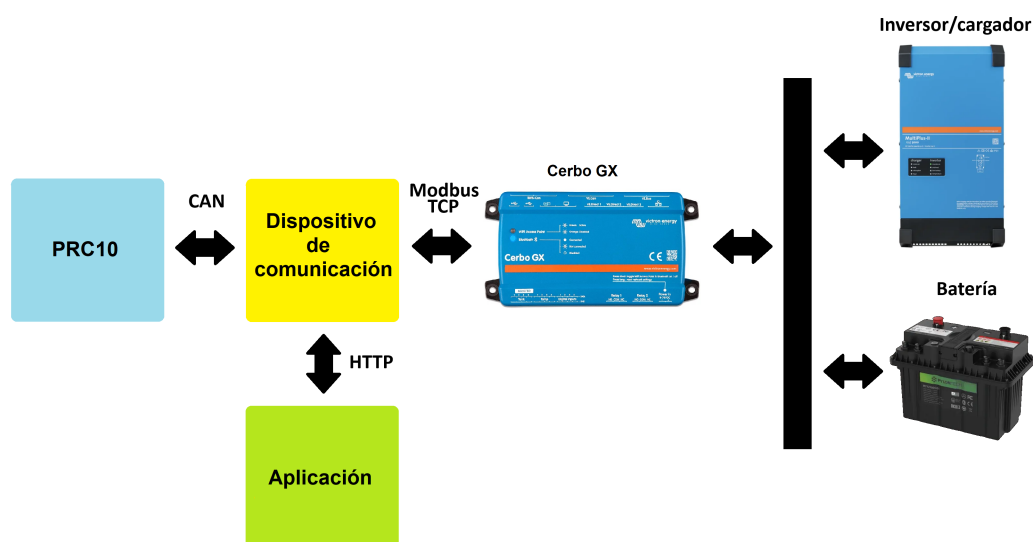


FIGURA 2.1. Arquitectura general del sistema.

2.2. Protocolos de comunicación

Como se mencionó en la sección 2.1, el dispositivo debía cumplir con la funcionalidad de actuar como intermediario entre las redes establecidas por Enertik y Victron, así como también constituir un punto de acceso para que la aplicación móvil opere sobre el sistema.

Para lograr este objetivo, fue necesario implementar distintos mecanismos de comunicación que permiten el intercambio de información entre los subsistemas involucrados. Estos mecanismos abarcan la interacción con la red de domótica existente, la comunicación con el ecosistema Victron y el acceso desde la aplicación móvil.

2.2.1. Protocolo CAN

El protocolo CAN (*Controller Area Network*) es un estándar de comunicación serie ampliamente utilizado en la industria automotriz, diseñado para el intercambio de información entre múltiples dispositivos en sistemas distribuidos [6].

Este protocolo emplea una transmisión diferencial sobre un par de conductores (CAN H y CAN L), lo que le otorga una elevada inmunidad a interferencias electromagnéticas y lo hace adecuado para entornos eléctricos exigentes.

En el presente trabajo, la incorporación del protocolo CAN se encuentra directamente vinculada a su utilización en la red de domótica desarrollada por Enertik. Dado que los dispositivos del sistema ya emplean este protocolo para su comunicación, su adopción permitió garantizar la compatibilidad con la infraestructura existente y evitar rediseñar los módulos involucrados.

En este contexto, las características del protocolo CAN, como su inmunidad al ruido eléctrico, su capacidad de comunicación entre múltiples nodos y su uso extendido en aplicaciones automotrices, lo convierten en una solución adecuada para su implementación en sistemas de domótica vehicular. Esto facilitó la integración del dispositivo desarrollado dentro de la red existente, sin necesidad de modificaciones en la infraestructura.

2.2.2. Protocolo Modbus TCP

El protocolo Modbus TCP es una variante del protocolo Modbus que opera sobre redes Ethernet mediante el uso de TCP/IP como medio de transporte [9]. Este protocolo permite el acceso a registros internos de dispositivos remotos a través de una estructura de comunicación simple y estandarizada.

En el presente trabajo, Modbus TCP se utiliza para establecer la comunicación con el módulo Cerbo GX. Este dispositivo expone distintos registros asociados al estado y funcionamiento de los equipos del ecosistema Victron [10]. A través de este protocolo, el dispositivo basado en ESP32 consulta variables relevantes y envía comandos de control.

La elección de Modbus TCP responde a que constituye el mecanismo de comunicación provisto de forma nativa por los equipos Victron. Esto permite acceder a la información del sistema y modificar su comportamiento mediante una interfaz documentada y estandarizada, sin necesidad de desarrollar soluciones propietarias. Además, su estructura basada en registros facilita la organización y el acceso a las variables de interés dentro del sistema.

2.2.3. Solicitudes HTTP

HTTP es un protocolo de aplicación ampliamente utilizado para la comunicación entre clientes y servidores. Su funcionamiento se basa en un modelo de tipo *request-response*, donde un cliente envía una solicitud (request) a un servidor, que procesa la petición y devuelve una respuesta (response) con la información requerida [11].

En el presente trabajo, se emplea como mecanismo de comunicación entre la aplicación móvil y el dispositivo basado en ESP32. A través de solicitudes HTTP, la aplicación envía comandos de control y solicita información del sistema, lo que permite la interacción remota con la red de domótica.

La elección de HTTP se fundamenta en su simplicidad de implementación y en su compatibilidad con entornos móviles. La comunicación se establece de forma directa entre la aplicación y el ESP32 dentro de la red local, sin intermediarios, lo que permite obtener tiempos de respuesta adecuados para la operación del sistema. En este esquema, el ESP32 actúa como servidor y procesa las solicitudes recibidas para generar las respuestas correspondientes, lo que simplifica el diseño del firmware y contribuye a un funcionamiento confiable del sistema.

2.3. Componentes de hardware

Para el desarrollo de hardware fue necesaria la utilización de ciertos componentes. El principal de ellos fue el ESP32 como microcontrolador del equipo. A su vez, para poder adaptarlo a los distintos protocolos de comunicación, fue necesario el agregado de dos periféricos adicionales: el módulo W5100 [12] y el MCP2551 [13].

2.3.1. Microcontrolador ESP32

El ESP32 es un microcontrolador desarrollado por la empresa Espressif Systems. Se trata de un dispositivo SoC (*System on Chip*) que integra múltiples periféricos y capacidades de procesamiento en un único dispositivo.

Entre sus principales características, el ESP32 dispone de puertos de propósito general (GPIO), conversores analógico-digitales (ADC), módulos de comunicación serie como UART (*Universal Asynchronous Receiver-Transmitter*) [14], SPI (*Serial Peripheral Interface*) [15] e I²C (*Inter-Integrated Circuit*) [16], temporizadores y periféricos para generación de señales PWM (*Pulse Width Modulation*) [17].

Además, el dispositivo incorpora conectividad inalámbrica, como Wi-Fi y Bluetooth, lo que lo convierte en una plataforma adecuada para el desarrollo de aplicaciones orientadas al campo IoT (Internet of Things).

La elección del ESP32 responde a la necesidad de disponer de un microcontrolador con capacidad para gestionar múltiples interfaces de comunicación y brindar conectividad inalámbrica, lo que permite su integración con los distintos subsistemas del vehículo y con la aplicación móvil. En este sentido, sus características lo convierten en una opción adecuada para el desarrollo del presente trabajo.



FIGURA 2.2. Microcontrolador ESP32¹.

2.3.2. Módulo transceptor

Todos los dispositivos que pertenecen a la red de domótica de Enertik se conectan a través de un cableado de cuatro conductores (+12 V, GND, CAN H y CAN L). Este esquema permite tanto la alimentación como la comunicación de los equipos conectados a la red de manera sencilla.

Por otra parte, el protocolo de comunicación empleado corresponde a una biblioteca propia desarrollada por Enertik, basada en el estándar CAN. Para adaptar el ESP32 a esta red y permitir su correcta integración, fue necesaria la implementación de un módulo transceptor.

Este módulo actúa como interfaz entre el ESP32 y la red de domótica, y está compuesto por un microcontrolador PIC [18], en el que se implementa la biblioteca de comunicación, y un transceptor CAN MCP2551, encargado de la adaptación de niveles eléctricos para la transmisión diferencial sobre el bus.

Como se puede apreciar en la figura 2.3, este módulo realiza la conversión de la red CAN, proveniente del sistema de domótica, a una interfaz de comunicación serial UART, compatible con el ESP32.

Esta solución permite desacoplar la gestión del protocolo de comunicación del procesamiento principal del sistema, lo que simplifica el desarrollo del firmware y facilita la integración del dispositivo en la red de domótica existente.



FIGURA 2.3. Funcionamiento del módulo transceptor.

¹Imagen tomada de <https://robu.in/product/esp32-38pin-development-board-wifiblueetooth-ultra-low-power-consumption-dual-core/>.

2.3.3. Módulo Ethernet W5100

Para establecer la comunicación con el sistema Victron mediante el protocolo Modbus TCP, se incorporó un módulo Ethernet basado en el circuito W5100.

El W5100 es un controlador de red que integra la pila de protocolos TCP/IP en hardware, lo que permite gestionar la comunicación Ethernet sin sobrecargar al microcontrolador principal. La interfaz con el ESP32 se realiza a través de SPI, lo que simplifica su integración a nivel de hardware.

En el presente trabajo, este módulo cumple la función de proporcionar una interfaz de red cableada para el acceso al módulo Cerbo GX, ya que la comunicación con los equipos Victron se realiza sobre una red Ethernet local.

La incorporación del W5100 permite delegar la gestión de la comunicación de red en un periférico dedicado, lo que reduce la carga de procesamiento sobre el ESP32 y facilita la implementación del protocolo Modbus TCP dentro del sistema. Asimismo, se trata de una solución de bajo costo y amplia disponibilidad en el mercado, lo que favorece su adopción en desarrollos embebidos y su posible integración en un producto comercial.



FIGURA 2.4. Módulo W5100².

2.4. Herramientas de software

Para el desarrollo del software fue necesario definir tanto el entorno de programación integrado como el lenguaje a emplear. Asimismo, se evaluaron las herramientas necesarias para la implementación de la aplicación móvil.

2.4.1. Entorno de desarrollo Arduino

El microcontrolador ESP32 es compatible con el entorno de desarrollo Arduino [19], lo que permitió utilizar este IDE (*Integrated Development Environment*) como plataforma principal para la implementación del firmware. La elección de este entorno se basó en su facilidad de uso, amplia documentación y la disponibilidad de una gran cantidad de bibliotecas previamente desarrolladas.

En particular, el uso del IDE Arduino facilitó la implementación de los distintos protocolos de comunicación requeridos en el trabajo, tales como HTTP y Modbus TCP. Para este último, se emplearon bibliotecas específicas que fueron adaptadas para su funcionamiento sobre el módulo Ethernet W5100.

²Imagen tomada de <https://grobotronics.com/wiznet-w5100-ethernet-network-module.html>.

Adicionalmente, este entorno permite una rápida compilación y carga de programas sobre el dispositivo, así como el uso de herramientas de depuración básicas, lo que lo convierte en una opción adecuada para el desarrollo y prueba del sistema.

Para la implementación de la comunicación Modbus TCP, se utilizó una biblioteca desarrollada por Andre Sarmiento Barbosa y Alexander Emelianov [20]. En conjunto con la biblioteca Ethernet provista por el entorno Arduino, esta herramienta permite la implementación del protocolo Modbus en distintas variantes, entre ellas Modbus TCP.

Para el desarrollo de la comunicación UART y del servidor interno en el ESP32, se emplearon las bibliotecas estándar provistas por defecto en el entorno de desarrollo.

2.4.2. Funcionamiento del ESP32

Para la programación del microcontrolador se optó por utilizar C++, debido a que es el lenguaje en el que se encuentran desarrolladas la mayoría de las bibliotecas disponibles en el entorno Arduino, lo que facilitó la integración de las distintas funcionalidades requeridas en el trabajo.

Para la implementación del firmware se utilizó FreeRTOS [21], disponible de forma nativa en el ESP32.

La adopción de este sistema operativo responde a la necesidad de gestionar múltiples tareas de manera concurrente dentro del sistema. En particular, el dispositivo debe atender de forma simultánea la comunicación con la red de domótica mediante CAN, el intercambio de información con el sistema Victron a través de Modbus TCP y la interacción con la aplicación móvil mediante HTTP.

La organización del firmware en tareas independientes permite desacoplar estas funcionalidades, de modo que cada una pueda ejecutarse con su propia lógica y requerimientos temporales, sin interferir con las demás. Esto resulta especialmente relevante en un sistema donde coexisten mecanismos de comunicación con diferentes tiempos de respuesta.

FreeRTOS incorpora un planificador (*scheduler*) que asigna el tiempo de procesamiento a cada tarea en función de su prioridad. Además, proporciona mecanismos de comunicación y sincronización, como colas y semáforos, que permiten coordinar el intercambio de información entre tareas de forma segura.

Este enfoque contribuye a una estructura de firmware más organizada y facilita la escalabilidad del sistema ante la incorporación de nuevas funcionalidades.

2.4.3. Desarrollo de la aplicación móvil

Para el desarrollo de la aplicación móvil se optó por utilizar un conjunto de herramientas que trabajan de forma integrada. La figura 2.5 presenta un esquema de la relación entre las distintas tecnologías empleadas.

Se utilizó el framework Ionic [22] como base para el desarrollo de la aplicación, debido a que permite construir interfaces móviles a partir de tecnologías web y mantener una única base de código.

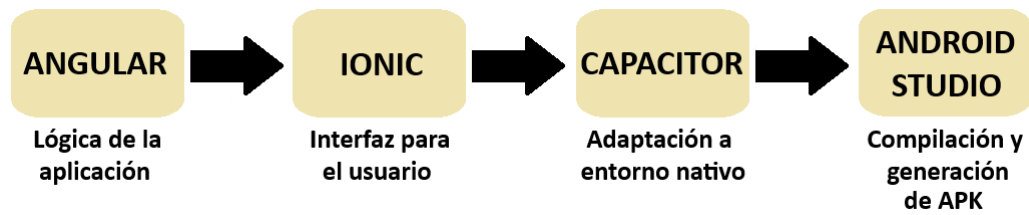


FIGURA 2.5. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.

Ionic se integra con Angular [23], framework que facilita la organización de la aplicación en componentes y la gestión del estado de la interfaz. El uso de tecnologías como TypeScript [24], HTML [25] y CSS [26] permitió estructurar la aplicación de forma modular y mantener una separación clara entre lógica y presentación.

Para la ejecución en dispositivos móviles se utilizó Capacitor [27], que permite empaquetar la aplicación como una aplicación nativa y acceder a funcionalidades propias del dispositivo cuando resulta necesario.

Finalmente, se empleó Android Studio [28] para la generación del archivo APK (*Android Package Kit*), necesario para la instalación de la aplicación en dispositivos Android.

La elección de este conjunto de herramientas responde a la necesidad de desarrollar una interfaz de usuario flexible y fácilmente mantenible, capaz de comunicarse con el dispositivo basado en ESP32 mediante solicitudes HTTP. Este enfoque permitió implementar rápidamente una solución funcional sin requerir el desarrollo de aplicaciones nativas desde cero.

Capítulo 3

Diseño e implementación

En este capítulo se describe el diseño del dispositivo, así como la arquitectura de hardware y software que lo componen, junto con el desarrollo de la aplicación destinada al control del motorhome.

3.1. Desarrollo de hardware

Como se mencionó en la sección 1.4, el microcontrolador ESP32 debe actuar como intermediario y vínculo entre las tres partes principales: módulo PRC10, Cerbo GX y aplicación móvil.

La figura 3.1 muestra una estructura del flujo o camino que debe seguir el ESP32 para llegar a cada una de estas partes.

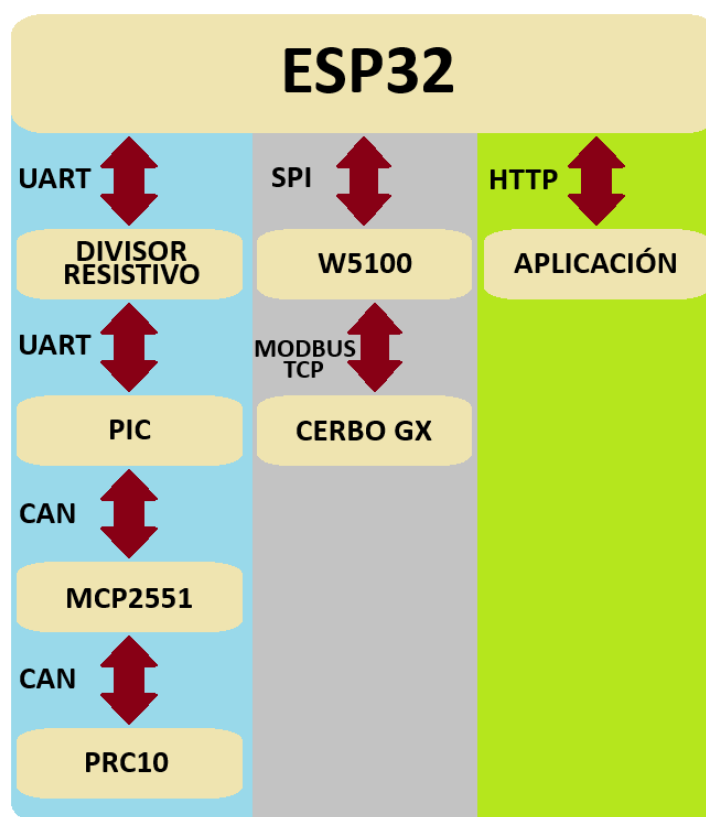


FIGURA 3.1. Esquema de interconexión entre el ESP32 y los módulos externos.

La columna celeste representa la forma de comunicar el ESP32 con el módulo PRC10. Asimismo, las columnas gris y verde representan los caminos hacia el Cerbo GX y aplicación móvil respectivamente.

3.1.1. Comunicación con la red Enertik

Como se mencionó en la sección 2.2.1, la red de domótica de Enertik utiliza una biblioteca propia basada en el protocolo CAN. A su vez, todos los dispositivos de la red se conectan a través de un cableado de cuatro conductores: dos destinados a la alimentación de los equipos y dos destinados a la comunicación CAN (CAN H y CAN L).

Para lograr una comunicación compatible y confiable entre el ESP32 y la red CAN del sistema, fue necesario incorporar un módulo transceptor provisto por Enertik, como se muestra en la figura 3.2.

Este módulo permite la conversión del protocolo CAN, proveniente de la red del motorhome, a una interfaz de comunicación UART compatible con el ESP32. De este modo, se garantiza la correcta integración con la infraestructura existente y se evita la necesidad de adaptar o replicar las configuraciones propias de la biblioteca desarrollada por la empresa.

Asimismo, esta solución simplifica la implementación de la comunicación, ya que delega el procesamiento específico del protocolo CAN en un dispositivo diseñado para tal fin.



FIGURA 3.2. Módulo transceptor comercial perteneciente a Enertik.

Basado en esta premisa, se decidió replicar y adaptar este módulo transceptor para que sea compatible con el dispositivo basado en ESP32. Este módulo se compone de un microcontrolador PIC y de un transceptor CAN MCP2551.

La figura 3.3 muestra un diagrama completo sobre cómo fue la implementación de este módulo transceptor.

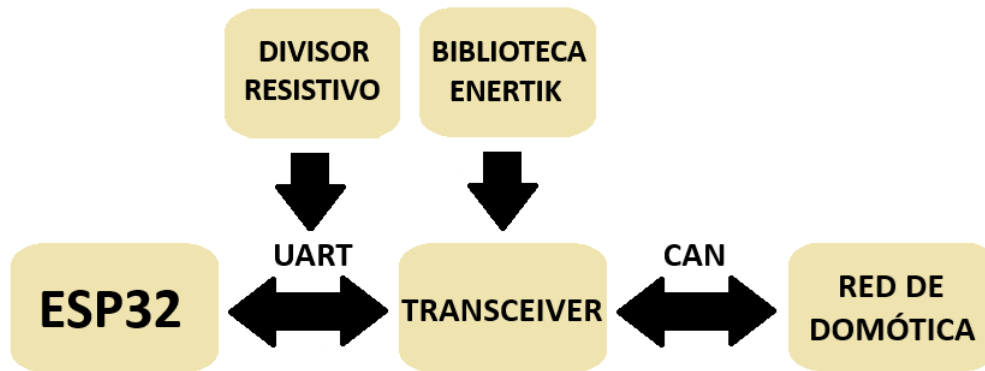


FIGURA 3.3. Diagrama de cómo se establece la comunicación CAN entre la red del motorhome y el ESP32.

El microcontrolador PIC dispone de la biblioteca de comunicación propia desarrollada por Enertik, lo que le permite interactuar de forma nativa con la red de domótica del motorhome. Por su parte, el transceptor MCP2551 se encarga de la adaptación de niveles eléctricos necesarios para la transmisión diferencial sobre el bus CAN.

El microcontrolador PIC funciona con una tensión de alimentación de 5 V, mientras que el ESP32 opera con 3,3 V. Esto implica que los niveles de tensión utilizados en los puertos UART de cada dispositivo son diferentes.

Esta diferencia generó la necesidad de adaptar los niveles de tensión en la comunicación UART, ya que una conexión directa podría dañar los puertos del ESP32 debido a su menor tolerancia de voltaje.

La solución adoptada fue la implementación de un divisor resistivo. La figura 3.4 muestra un diagrama del circuito utilizado, junto con el cálculo correspondiente a la tensión de salida.

Mediante el uso de tres resistencias de igual valor, se logró reducir la tensión de 5 V a 3,3 V en la línea de transmisión hacia el ESP32, lo que permitió una comunicación segura entre ambos dispositivos.

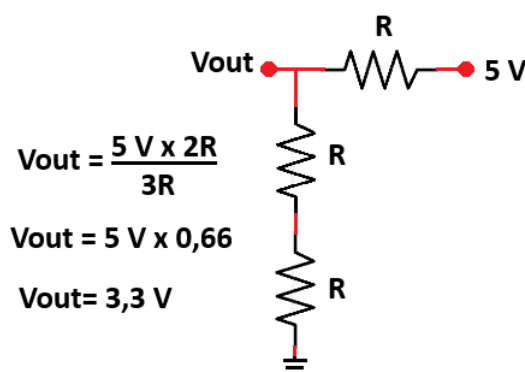


FIGURA 3.4. Diseño de divisor resistivo para adaptación UART.

Este divisor fue implementado en la conexión correspondiente a la línea de transmisión UART (Tx) del microcontrolador PIC hacia la línea de recepción UART (Rx) del ESP32. En el sentido contrario no fue necesaria esta adaptación, ya que el nivel lógico de 3,3 V proveniente del ESP32 es reconocido correctamente por el PIC.

De esta manera, el módulo transceptor actúa como un intermediario que realiza la conversión entre la red CAN y una interfaz UART. Esto permite al ESP32 enviar y recibir información de la red de domótica sin necesidad de implementar directamente la complejidad del protocolo CAN utilizado por Enertik.

3.1.2. Comunicación con la red Victron

La comunicación con la red Victron se realizó mediante el protocolo Modbus TCP. Desde el punto de vista del hardware, el microcontrolador ESP32 no dispone de una interfaz Ethernet integrada, por lo que fue necesario incorporar un módulo externo que permita este tipo de conectividad.

Durante la etapa de investigación se analizaron distintas alternativas para implementar la comunicación Ethernet. Como resultado, se optó por utilizar el módulo W5100, debido a su amplia disponibilidad en el mercado, su bajo costo y su compatibilidad con entornos de desarrollo basados en Arduino.

Adicionalmente, el IDE Arduino dispone de bibliotecas específicas para la comunicación Ethernet que resultan compatibles con este módulo, lo que facilitó su integración dentro del sistema. Estas características hicieron del módulo W5100 una opción adecuada para la implementación de la comunicación Modbus TCP con los equipos del ecosistema Victron.

Otra ventaja de este componente es que utiliza el protocolo de comunicación SPI. El ESP32 dispone de periféricos compatibles con este protocolo, lo que facilitó su integración dentro del sistema.

Más allá del conexionado SPI (pines MOSI, MISO, SCK y CS), la conexión entre el ESP32 y el módulo W5100 no requirió hardware adicional.

Este módulo funciona con una tensión de alimentación de 5 V, lo que lo vuelve compatible con el diseño implementado, ya que se aprovecha la misma fuente utilizada para energizar el microcontrolador PIC del módulo transceptor.

La conexión Ethernet que parte desde el módulo W5100 se realiza a través de un conector RJ45 [29]. Este conector permite la utilización de cables de red estándar, compuestos por pares trenzados de conductores, diseñados para la transmisión de datos a alta velocidad.

Estos cables emplean una transmisión diferencial sobre cada par, lo que reduce la susceptibilidad a interferencias electromagnéticas y mejora la integridad de la señal. De esta manera, se garantiza una comunicación confiable entre el dispositivo desarrollado y la red del sistema Victron.

Mediante este tipo de conexión, el módulo W5100 puede integrarse directamente a la red Ethernet donde se encuentra el módulo Cerbo GX, lo que permite el intercambio de datos necesario para la implementación del protocolo Modbus TCP.

Para lograr la comunicación, resta definir qué dispositivo cumplirá el rol de maestro y cuál el de esclavo. Esta definición se abordará en la sección 3.2, donde se detallará su implementación.

3.1.3. Desarrollo de la placa final y listado de componentes

El diseño del PCB (*Printed Circuit Board*) se desarrolló mediante el software Altium Designer [30]. Este software es ampliamente utilizado en la industria, ya que permite crear esquemáticos de circuitos de forma sencilla. Estos sirven como base para el diseño de placas de circuito impreso.

Para la creación de este PCB, se tomó como referencia el conexionado utilizado en el prototipo experimental (este se explicará en la sección 4.4.1).

Dado que se espera que este desarrollo se convierta en un producto comercial, fue necesario realizar modificaciones en la selección de componentes durante el diseño de este prototipo.

La primera diferencia se encuentra en el microcontrolador ESP32. Para el prototipo experimental y los ensayos iniciales se utilizó una placa de desarrollo comercial basada en este dispositivo.

En el diseño del PCB final, se optó por reemplazar dicha placa por el chip del microcontrolador en sí. Esta decisión permitió descartar los componentes adicionales presentes en la placa de desarrollo. De esta forma, se eliminó hardware innecesario, se redujeron costos y se optimizó el espacio disponible en el PCB.

Por otra parte, fue necesaria la incorporación de un regulador de tensión AP63203 [31]. Este dispositivo permite adaptar el nivel de tensión de 12 V proveniente de la red del motorhome a un valor de 3,3 V, compatible con el chip ESP32.

En el caso del módulo transceptor, dado que se conocía completamente su diseño, se optó por integrarlo directamente en el PCB. Para ello, se incorporaron el microcontrolador PIC18F26K80 y el transceptor MCP2551 en la placa. Esta decisión permitió optimizar el espacio disponible y reducir costos al unificar todos los componentes en una única placa.

En cuanto al módulo W5100, se optó por utilizar la versión comercial disponible en el mercado. Como consecuencia, en el PCB se reservó un espacio para la colocación de un conector compatible con el módulo W5100, lo que permite su montaje manual de forma sencilla.

En la figura 3.5 se puede observar el diseño final del PCB.

Como se puede apreciar, el PCB está compuesto por los siguientes componentes:

- Componente 1: conector principal. Terminal sobre el que se conecta el cableado de cuatro hilos proveniente del motorhome.
- Componente 2: MCP2551. Forma parte del módulo transceptor, encargado de adaptar los niveles de tensión de la línea CAN.
- Componente 3: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceptor.

- Componente 4: PIC18F26K80. Microcontrolador que contiene la biblioteca desarrollada por Enertik para la comunicación entre equipos.
- Componente 5: módulo ESP32. Microcontrolador que contiene el firmware principal del equipo.
- Componente 6: regulador de tensión AP63203. Fuente de tensión de 3,3 V encargado de alimentar el ESP32.
- Componente 7: regulador de tensión 7805 [32]. Componente responsable de suministrar una tensión estable de 5 V para alimentar el PIC y el W5100.
- Componente 8: conector de diez pines. Conector compatible con el módulo W5100. En este lugar se montará dicho componente.

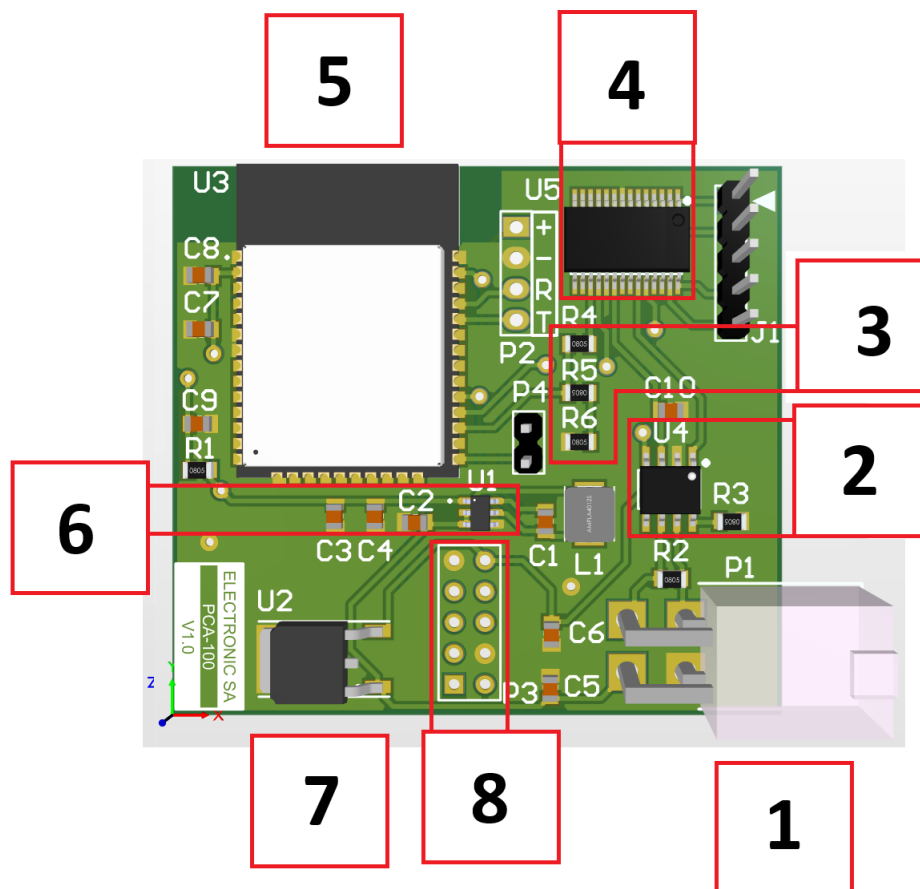


FIGURA 3.5. Diseño final para PCB del equipo.

Para el proceso de fabricación, fue necesaria la contratación de servicios de un fabricante especializado en este campo. Para ello, se generaron los denominados archivos Gerber [33].

3.2. Desarrollo de firmware

Como se mencionó en la sección 2.4.2, se optó por utilizar FreeRTOS como base para el desarrollo del firmware.

FreeRTOS permite estructurar el firmware mediante la ejecución de múltiples tareas. Su planificación genera un comportamiento concurrente a nivel lógico, lo que mejora la organización del sistema y facilita la gestión de sus funcionalidades.

En este contexto, para el presente trabajo se implementaron seis tareas principales, ilustradas en la figura 3.6, donde se aprecian las relaciones existentes entre ellas y con los módulos externos.

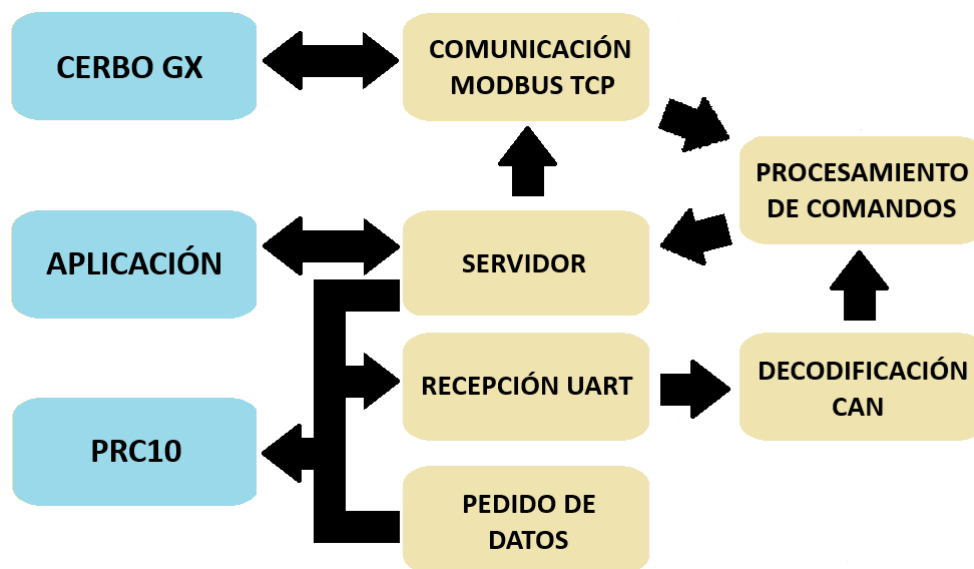


FIGURA 3.6. Relación entre las distintas tareas del firmware y su comunicación con el exterior.

3.2.1. Tarea: recepción UART

Esta tarea se encarga de monitorear de manera continua el puerto UART del ESP32. Cada vez que se detecta una señal en sus puertos, la tarea almacena la información recibida en un buffer de tipo circular [34].

Un buffer circular, también conocido como *ring buffer*, es una estructura de datos que utiliza una región de memoria de tamaño fijo y organiza los datos de forma continua. Cuando se alcanza el final del buffer, la escritura continúa desde el inicio, lo que permite reutilizar la memoria de manera eficiente sin necesidad de realizar desplazamientos de datos.

Se optó por este tipo de estructura porque resulta adecuada para la recepción de datos en tiempo real, ya que permite desacoplar la velocidad de llegada de la información respecto de su procesamiento. De esta forma, se evita la pérdida de datos ante ráfagas de información y se garantiza un manejo más robusto de la comunicación UART.

3.2.2. Tarea: decodificación de comando CAN

Esta tarea se encarga de decodificar los comandos recibidos desde la red CAN. Para ello, realiza la lectura de los datos almacenados en el buffer circular.

La información transmitida en la red CAN sigue un formato predefinido. El primer byte corresponde a una marca de inicio, que permite identificar el comienzo de cada trama. Por otro lado, el último byte corresponde a un valor de control CRC (*Cyclic Redundancy Check*), utilizado para verificar la integridad de los datos recibidos mediante un cálculo basado en el contenido del mensaje.

La trama incluye un campo de dirección de origen para identificar al dispositivo emisor, un campo de dirección de destino para indicar el dispositivo receptor, un campo de comando asociado a la acción a ejecutar y un conjunto de bytes de datos.

El análisis de cada trama completa se realiza mediante el recorrido del buffer circular hasta encontrar el byte de marca de inicio.

A continuación, extrae la trama completa y realiza el cálculo del CRC, lo que permite determinar si la trama es válida o no. En caso afirmativo, se procede a identificar los campos que componen el mensaje.

Cuando se detecta una trama válida, la información útil se transfiere a la siguiente tarea, encargada de procesar la instrucción correspondiente.

3.2.3. Tarea: comunicación Modbus TCP

Esta tarea se encarga de gestionar la comunicación Modbus TCP y, por lo tanto, es responsable del control del módulo Cerbo GX.

Para establecer la comunicación mediante este protocolo, fue necesario realizar una serie de configuraciones iniciales.

Se definió que el ESP32 ejerza el rol de maestro, encargado de solicitar datos y enviar comandos al Cerbo GX.

La comunicación entre ambos dispositivos se realiza a través de un cable Ethernet con conector RJ45, que vincula el Cerbo GX con el módulo W5100 controlado por el ESP32.

Para que esta comunicación sea posible, fue necesario definir direcciones IP estáticas para ambos dispositivos. Esto permite que cada equipo conozca la dirección del otro y mantenga una comunicación consistente en el tiempo. Además, ambos deben pertenecer a la misma red, lo que implica compartir la misma máscara de subred y una configuración de puerta de enlace (gateway) compatible.

La dirección IP del Cerbo GX se configura desde el menú del propio equipo. Por otro lado, la dirección IP del ESP32 se definió por software dentro de esta tarea.

Para su funcionamiento, se utiliza un contador interno que cumple la función de temporizador. En primer lugar, establece la comunicación Modbus TCP y se la mantiene activa durante el funcionamiento del sistema.

Posteriormente, en intervalos periódicos, realiza la lectura de los registros que el Cerbo GX pone a disposición para la obtención de datos y el control del comportamiento del equipo.

En este contexto, los parámetros de interés fueron aquellos asociados al modo de operación del equipo, ya sea en modo inversor o cargador.

Victron dispone de un registro que adopta distintos valores según el estado del equipo: modo inversor, modo cargador, funcionamiento combinado o estado apagado. A partir de la lectura de este registro es posible determinar el estado del equipo en todo momento.

Además, este registro admite operaciones de lectura y escritura, lo que permite modificar el comportamiento del equipo mediante la asignación de valores adecuados.

Otros parámetros relevantes incluyen la potencia suministrada por el equipo en modo inversor y la corriente de carga de entrada en modo cargador.

3.2.4. Tarea: pedido de datos

Esta tarea se encarga de solicitar de forma periódica información al módulo PRC10.

Dentro de la red de Enertik, el módulo PRC10 actúa como unidad central de procesamiento. En este módulo se concentran los valores y estados de todas las variables del motorhome.

Para que la aplicación pueda reflejar esta información en pantalla, resulta necesario que el ESP32 tenga acceso a dichos datos.

Con este objetivo, se realizó una modificación en el firmware del módulo PRC10. Se incorporó una función que permite generar una serie de paquetes, cada uno asociado a un conjunto de variables del sistema.

Entre las variables incluidas en estos paquetes se encuentran:

- Tensión de batería.
- Corriente suministrada por la batería.
- Tensión de entrada en alterna.
- Frecuencia de la tensión de entrada.
- Origen de la tensión de entrada (red domiciliaria, generador o inversor).
- Nivel de llenado de los tanques de agua (limpia, gris y negra).
- Estado del cargador.
- Corriente de carga.
- Estado del inversor.
- Potencia suministrada por el inversor.
- Estado de los pulsadores asociados a las distintas pantallas de la HMI.

Desde el lado del ESP32, esta tarea realiza solicitudes periódicas de cada uno de estos paquetes. El funcionamiento se basa en un ciclo en el que, cada 500 ms, se solicita un paquete distinto al PRC10.

Una vez completado el recorrido de todos los paquetes, el proceso se reinicia desde el primero. Este comportamiento cíclico permite mantener actualizada la información correspondiente a todas las variables del sistema.

3.2.5. Tarea: procesamiento de comandos recibidos

Esta tarea constituye el núcleo del firmware. Su función principal es recibir los comandos provenientes de las tareas de comunicación CAN (3.2.2) y Modbus TCP (3.2.3), procesarlos y actualizar las variables internas del sistema.

En relación con la comunicación CAN, esta tarea recibe las tramas ya validadas por la etapa de decodificación. A partir de estas, extrae los distintos campos de datos y los interpreta según el tipo de paquete recibido. Los valores obtenidos se almacenan en estructuras de datos internas, que representan el estado actualizado de los distintos sensores y actuadores del motorhome.

Adicionalmente, se realizan conversiones sobre los datos recibidos, ya que muchos de ellos se encuentran codificados. Estas conversiones permiten obtener magnitudes físicas en unidades reales, lo que facilita su posterior visualización y procesamiento.

Por otra parte, la tarea también gestiona comandos destinados al módulo Cerbo GX a través de Modbus TCP. En función del comando recibido, se determinan las acciones a ejecutar, como la activación o desactivación del inversor o del cargador. Para ello, se realiza la escritura sobre los registros correspondientes del dispositivo Victron, lo que permite modificar su estado de operación.

Finalmente, una vez procesada la información, los datos actualizados se envían a otras tareas del sistema a través de colas de FreeRTOS. Este mecanismo permite desacoplar el procesamiento de datos de su consumo, lo que mejora la organización y la escalabilidad del firmware.

3.2.6. Tarea: control de servidor

Esta tarea gestiona la implementación del servidor en el ESP32.

En primer lugar, se encarga de configurar el dispositivo en modo *access point*. En esta modalidad, el microcontrolador genera su propia red Wi-Fi, lo que permite la conexión directa de otros dispositivos. Además, se definen los parámetros de la red, tales como el nombre (SSID) y la contraseña de acceso.

Una vez establecida la conectividad, se definió la estructura del servidor HTTP mediante la creación de distintos endpoints. Estos representan puntos de acceso que permiten a los clientes interactuar con el sistema. Cada endpoint se asocia a una ruta específica y a un tipo de solicitud, como GET o POST, lo que permite consultar información o enviar comandos.

Se optó por implementar un endpoint para cada una de las pantallas principales de la aplicación. Este enfoque permite que cada pantalla solicite únicamente la información necesaria, lo que reduce el tráfico de datos y mejora la eficiencia del sistema.

En particular, se definieron cuatro endpoints que admiten solicitudes de tipo GET: home, luces, motores y energía.

Cada uno de estos endpoints genera y devuelve un archivo en formato JSON (*JavaScript Object Notation*) [35] con la información requerida por la pantalla correspondiente, lo que permite actualizar la interfaz de usuario con los valores actuales del sistema.

Por otra parte, la aplicación móvil dispone de distintos pulsadores que permiten controlar el funcionamiento del motorhome. Para ello, se implementó un quinto endpoint (denominado pulsadores) que admite solicitudes de tipo POST, destinado a la recepción de comandos.

Este endpoint recibe un archivo JSON que contiene dos campos: uno que identifica el pulsador que originó la acción y otro que indica el nuevo estado asociado.

A partir de esta información, el servidor interpreta el comando y lo traduce en una instrucción que es enviada al módulo PRC10 para así permitir la ejecución de la acción correspondiente dentro del sistema.

De esta manera, el servidor implementado en el ESP32 actúa como intermediario entre la aplicación móvil y la red de domótica, lo que permite centralizar tanto la consulta de información como el envío de comandos.

3.3. Desarrollo de la aplicación

Para el desarrollo de la aplicación se optó por utilizar el framework Ionic. Su elección aportó diversas ventajas durante el proceso de desarrollo.

En primer lugar, permitió utilizar el navegador web de la computadora como herramienta de depuración, lo que facilitó la visualización y prueba de la aplicación en tiempo real. Esto redujo significativamente los tiempos de desarrollo, ya que cada modificación realizada podía verificarse de forma inmediata.

Asimismo, este entorno facilitó la realización de ensayos sobre los endpoints definidos en el servidor del ESP32. Para ello, el dispositivo se conectó a la misma red Wi-Fi que la computadora de desarrollo, lo que permitió que la aplicación realizara solicitudes HTTP de tipo GET y POST hacia los endpoints correspondientes.

Estas solicitudes se dirigieron a la dirección IP asignada al ESP32, dato que el propio dispositivo informa al momento de establecer la conexión a la red.

Para el desarrollo de la aplicación, se tomó como referencia el software utilizado en la pantalla HMI del motorhome. Se replicaron la mayor cantidad posible de sus funcionalidades, con prioridad en aquellas consideradas de mayor importancia.

Como se mencionó en la sección 3.2.6, se optó por desarrollar una aplicación compuesta por cuatro pestañas. A su vez, se implementó una barra de navegación por pestañas que permite acceder a las distintas pantallas de la aplicación.

Previo a la implementación, se definió un criterio de diseño orientado a la organización modular de la interfaz. Para ello, se adoptó un enfoque basado en contenedores visuales personalizados, cuyo tamaño se define en términos relativos al espacio disponible en pantalla.

La figura 3.7 ilustra este criterio. Cada contenedor puede ocupar un porcentaje del ancho y del alto de la pantalla, lo que permite estructurar la interfaz mediante una disposición en filas y columnas.

A partir de este esquema, las pantallas se construyen como una composición de contenedores anidados. Dentro cada bloque final se integra cada uno de los elementos funcionales de la aplicación, tales como indicadores, controles y visualizaciones de datos.



FIGURA 3.7. Contenedores dentro de una pantalla.

Este enfoque responde a un modelo de layout basado en grillas, ampliamente utilizado en el desarrollo de interfaces modernas. Su uso permitió mantener una distribución consistente de los elementos en todas las pantallas y facilitó la adaptación del diseño a distintos tamaños de dispositivo.

En el ejemplo de la figura 3.7, se observa un contenedor principal que define el área de trabajo de la pantalla. A partir de este, se establecen subdivisiones en filas y columnas mediante nuevos contenedores, lo que permite organizar la ubicación de cada elemento de la interfaz.

Inicialmente, estos contenedores se representaron con colores para facilitar la visualización de la estructura. Una vez definida la disposición final, se ajustó su apariencia según el diseño de cada pantalla y, en la mayoría de los casos, se utilizó un fondo transparente para integrarlos visualmente con la interfaz.

3.3.1. Arquitectura funcional de la aplicación

La aplicación móvil se diseñó con una estructura basada en navegación por pestañas, con el objetivo de organizar las distintas funciones del sistema en módulos independientes.

Como se mencionó en la sección 3.2.6, se definieron cuatro vistas principales: home, luces, motores y energía. Cada una de ellas agrupa funcionalidades asociadas a un subsistema específico del motorhome, lo que simplifica la navegación y mejora la experiencia de uso.

La figura 3.8 presenta el mapa de navegación de la aplicación. En ella se observan las distintas vistas definidas y los principales elementos funcionales asociados a cada una de ellas, entre los que se incluyen variables de monitoreo, indicadores y controles.

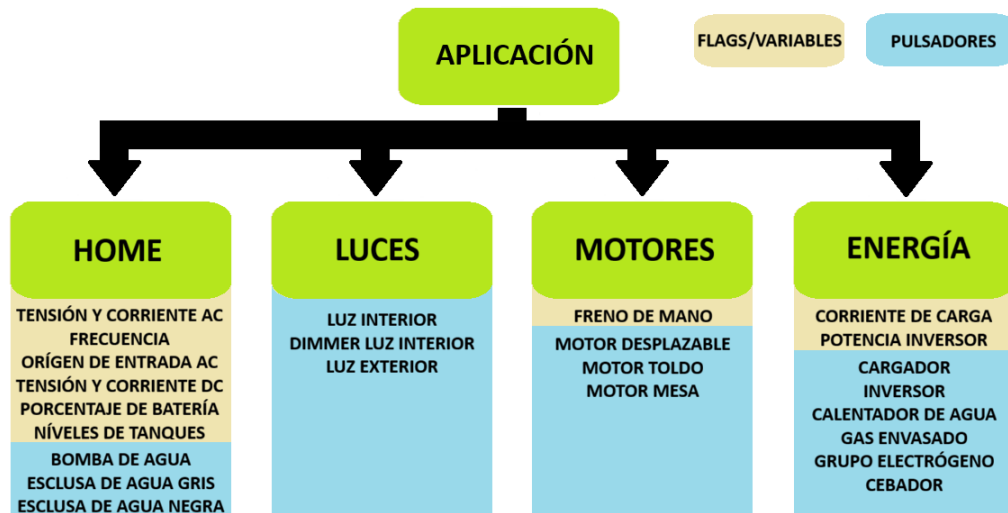


FIGURA 3.8. Arquitectura funcional de la aplicación.

La vista home constituye la pantalla principal de la aplicación y centraliza las variables más relevantes del sistema. En ella se incluyen magnitudes asociadas al sistema de baterías, variables de corriente alterna, niveles de los tanques de agua y controles correspondientes a actuadores de uso frecuente, como la bomba de agua y las esclusas.

La vista luces concentra las funciones asociadas al sistema de iluminación del motorhome. Permite supervisar el estado de los circuitos de iluminación interior y exterior, así como regular la intensidad de la iluminación interior mediante un control de tipo dimmer.

La vista motores agrupa el control de los elementos móviles del motorhome, tales como la mesa, el toldo y la cabina desplazable. Además, incorpora restricciones lógicas de seguridad vinculadas al estado del freno de mano del vehículo.

Finalmente, la vista energía centraliza el monitoreo y control de los principales subsistemas energéticos, entre ellos el grupo electrógeno, el cargador, el inversor y otros consumos de alta potencia vinculados al sistema.

3.3.2. Secuencia de interacción entre la aplicación y el servidor

El funcionamiento de la aplicación se basa en una arquitectura cliente-servidor, en la que el dispositivo móvil actúa como cliente y el ESP32 como servidor encargado de procesar las solicitudes y administrar el acceso a las variables del sistema.

La comunicación entre ambos dispositivos se implementó mediante el protocolo HTTP. En este esquema, la actualización de variables se realiza a través de solicitudes de tipo GET, mientras que las acciones de control se ejecutan mediante solicitudes de tipo POST.

La figura 3.9 representa la secuencia de intercambio correspondiente a la actualización periódica de variables.

Cada vez que el usuario accede a una pestaña de la aplicación, se activa un temporizador interno configurado con un período de tres segundos.

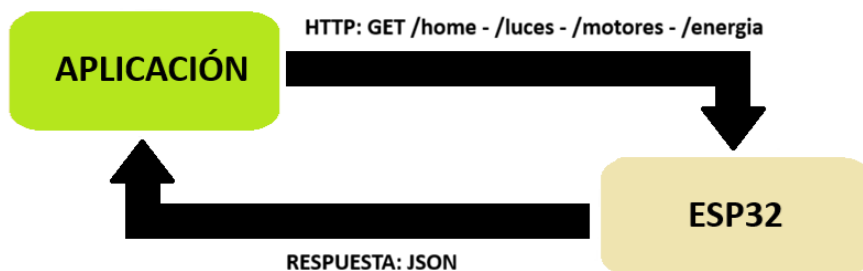


FIGURA 3.9. Secuencia de solicitudes GET entre la aplicación y el microcontrolador.

Al finalizar cada período, la pestaña activa envía una solicitud GET a su endpoint asociado. El ESP32 procesa dicha solicitud y responde con un archivo en formato JSON que contiene las variables correspondientes a esa vista, junto con el estado actual de sus controles.

Este mecanismo permite mantener sincronizada la información mostrada en pantalla con el estado real del sistema.

Por otra parte, las acciones de control se realizan mediante solicitudes HTTP de tipo POST. La figura 3.10 representa esta secuencia de comunicación.

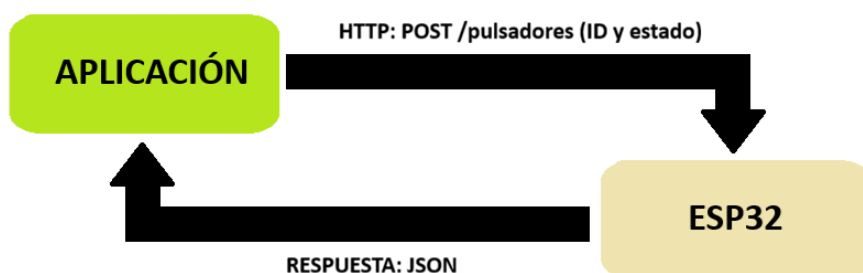


FIGURA 3.10. Secuencia de solicitudes POST entre la aplicación y el microcontrolador.

Cada vez que el usuario acciona un pulsador desde la interfaz, la aplicación genera una solicitud POST dirigida al endpoint pulsadores. En esta solicitud se transmite la identificación del control accionado y el nuevo estado requerido.

Luego, el ESP32 procesa la petición, ejecuta la lógica de control correspondiente y envía una respuesta de confirmación en formato JSON. Esta respuesta informa el nuevo estado del elemento controlado y permite mantener la coherencia entre el sistema físico y la interfaz de usuario.

3.3.3. Generación de instalador para Android

Una vez finalizado el desarrollo de la aplicación, fue necesario generar un instalador que permita su ejecución en dispositivos móviles con sistema operativo Android. Para ello, se utilizó la herramienta Capacitor, que posibilita la adaptación de aplicaciones desarrolladas con tecnologías web a un entorno nativo.

Capacitor actúa como una capa de integración entre la aplicación desarrollada con Ionic y el sistema operativo del dispositivo. A través de esta herramienta, la aplicación se empaqueta en una estructura compatible con Android. Además, brinda acceso a funcionalidades propias del dispositivo.

En este contexto, se emplearon herramientas provistas por Capacitor para la gestión de solicitudes HTTP desde la aplicación, y permitir así un control más directo sobre la comunicación con el servidor del ESP32. Asimismo, se utilizaron utilidades para la generación y configuración de los recursos gráficos de la aplicación, como los íconos y elementos visuales requeridos por el sistema Android.

A partir de este proceso, el trabajo fue importado en el entorno de desarrollo Android Studio. Este software permite gestionar la compilación de la aplicación, así como configurar parámetros propios del sistema Android, tales como permisos, versión mínima del sistema operativo y opciones de despliegue.

Como resultado de la compilación, se obtiene un archivo en formato APK, que constituye el formato estándar de distribución de aplicaciones para el sistema operativo Android. Este archivo contiene todos los recursos necesarios para la instalación de la aplicación, tales como el código ejecutable, las bibliotecas y los archivos de configuración.

La instalación del APK puede realizarse directamente sobre un dispositivo móvil mediante distintas alternativas. En este trabajo se utilizó la herramienta ADB (*Android Debug Bridge*) [36], que permite la transferencia e instalación del archivo desde una computadora hacia el dispositivo a través de una conexión USB. Una vez instalado, el usuario puede ejecutar la aplicación de forma independiente, sin necesidad de herramientas de desarrollo adicionales.

Capítulo 4

Ensayos y resultados

En este capítulo se explica cómo fueron los ensayos que se realizaron sobre el sistema y qué resultados se obtuvieron a partir de ellos.

4.1. Diseño final de la aplicación

Como resultado del proceso de desarrollo descrito en la sección 3.3, se obtuvo una aplicación móvil funcional orientada al monitoreo y control de distintos subsistemas del motorhome.

La interfaz final quedó organizada en cuatro pestañas principales, donde cada una de ellas fue diseñada para centralizar funciones específicas del sistema y facilitar la interacción del usuario con los distintos módulos controlados por el ESP32.

La figura 4.1 muestra el diseño final de la pantalla principal. La interfaz se organizó en bloques funcionales que agrupan las variables eléctricas del sistema, los niveles de los tanques de agua y los controles asociados. Esta disposición facilita la interpretación rápida del estado general del motorhome.



FIGURA 4.1. Pestaña home de la aplicación.

La figura 4.2 presenta la pantalla destinada al control de iluminación. La interfaz se organizó de forma diferenciada para los circuitos de iluminación interior y exterior e incorpora un control deslizante para el ajuste de intensidad de la luz interior. Esta disposición permite una operación simple e intuitiva.

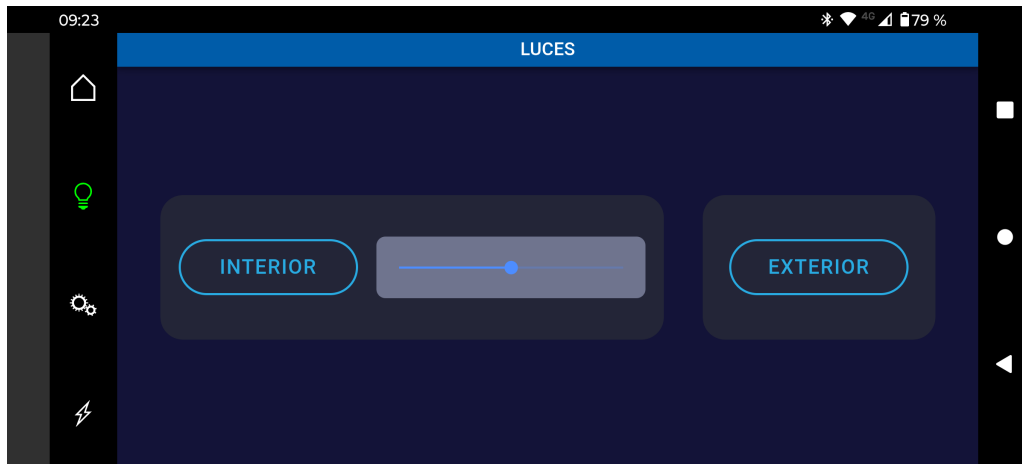


FIGURA 4.2. Pestaña luces de la aplicación.

La figura 4.3 muestra la pantalla destinada al control de los elementos móviles del motorhome. La interfaz se diseñó a partir de una estructura reutilizable, donde el usuario selecciona el elemento a controlar y accede a sus comandos asociados.

Además, se incorporaron indicadores visuales que informan el estado de habilitación de determinados actuadores, de acuerdo con las restricciones de seguridad definidas para el sistema.



FIGURA 4.3. Pestaña motores de la aplicación.

La figura 4.4 presenta la pantalla destinada al control energético. La interfaz se organizó en bloques funcionales independientes, lo que facilita la identificación de cada subsistema y permite supervisar múltiples equipos de forma simultánea.

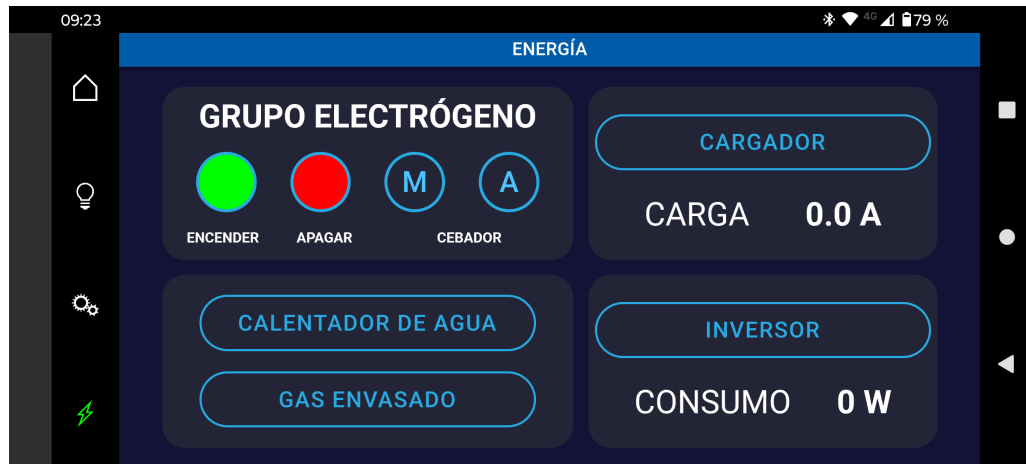


FIGURA 4.4. Pestaña energía de la aplicación.

Estas pantallas constituyen la versión final utilizada durante los ensayos presentados en las secciones siguientes.

4.2. Ensayos sobre la aplicación

En esta etapa se realizaron ensayos sobre la aplicación móvil de forma independiente, con el objetivo de validar su funcionamiento antes de su integración con el sistema embebido.

Para ello, se utilizó el entorno de desarrollo provisto por Ionic mediante el comando `ionic serve`, que permite ejecutar la aplicación en un navegador web. Este entorno facilitó el acceso a herramientas de depuración, tales como la simulación de distintos tamaños de pantalla y la visualización de mensajes en la consola.

La figura 4.5 muestra el entorno de ejecución utilizado durante los ensayos.



FIGURA 4.5. Entorno de ejecución y depuración de la aplicación.

En el sector identificado como cuadro 1, se empleó la funcionalidad de simulación de distintos tamaños de pantalla, lo que permitió evaluar el comportamiento de la interfaz frente a diferentes resoluciones.

Por otro lado, en la figura 4.6 se muestra la consola del navegador (indicada con el cuadro 2), donde se visualizaron mensajes de depuración generados mediante instrucciones como `console.log()`. Estos mensajes incluyeron valores de variables y estructuras de datos en formato JSON, lo que facilitó el seguimiento del estado interno de la aplicación y la detección de errores durante su ejecución.



FIGURA 4.6. Consola del navegador utilizada como depurador.

El procedimiento de ensayo consistió en navegar entre las distintas pestañas de la aplicación, accionar los controles disponibles y observar el comportamiento de la interfaz. En paralelo, se analizaron los mensajes generados en la consola del navegador mediante instrucciones de depuración incorporadas en el código.

Adicionalmente, se implementó un esquema de prueba basado en la modificación manual de variables internas (*hardcodeo*), con el fin de simular distintos estados del sistema. Esto permitió evaluar la forma en que la aplicación representa en pantalla valores correspondientes a diferentes condiciones de operación.

Como resultado, se verificó que la navegación entre pantallas se realizaba correctamente, que los controles respondían a las acciones del usuario y que los indicadores visuales reflejaban de manera coherente los valores asignados durante las pruebas.

Asimismo, se comprobó que la interfaz mantenía un comportamiento consistente frente a distintos tamaños de pantalla, sin presentar errores de visualización ni desajustes en la distribución de los elementos.

A partir de estos ensayos, se concluyó que la aplicación presentaba un funcionamiento adecuado a nivel de interfaz y lógica interna, lo que permitió avanzar hacia su integración con el sistema basado en ESP32.

4.3. Ensayos sobre el sistema

En esta etapa se realizaron ensayos de integración entre la aplicación móvil y el sistema basado en ESP32, con el objetivo de validar la comunicación entre ambos y el correcto funcionamiento del servidor implementado en el microcontrolador.

Para ello, se utilizó el entorno de ejecución de la aplicación en el navegador web, junto con el monitor serie del IDE Arduino, empleado para visualizar mensajes de depuración generados por el firmware del ESP32.

El montaje de ensayo consistió en conectar el ESP32 a una red Wi-Fi local, compartida con la computadora en la que se ejecutaba la aplicación.

Al establecer la conexión, el microcontrolador informó su dirección IP a través del monitor serie, lo que permitió acceder a los endpoints del servidor desde la aplicación.

El procedimiento de prueba incluyó el acceso a los distintos endpoints mediante solicitudes HTTP de tipo GET, así como el envío de comandos a través de solicitudes POST generadas desde la interfaz.

Con el fin de evaluar la comunicación, se implementó en el firmware una función temporal que asignaba valores aleatorios a las variables contenidas en las estructuras JSON. Estos valores eran informados mediante mensajes en el monitor serie.

A partir de este esquema, se compararon los valores reportados por el ESP32 con los recibidos por la aplicación, y se verificó su coincidencia junto con su correcta representación en la interfaz.

La figura 4.7 muestra un ejemplo del archivo JSON recibido desde el endpoint home, que contiene tanto variables del sistema como estados de los pulsadores.

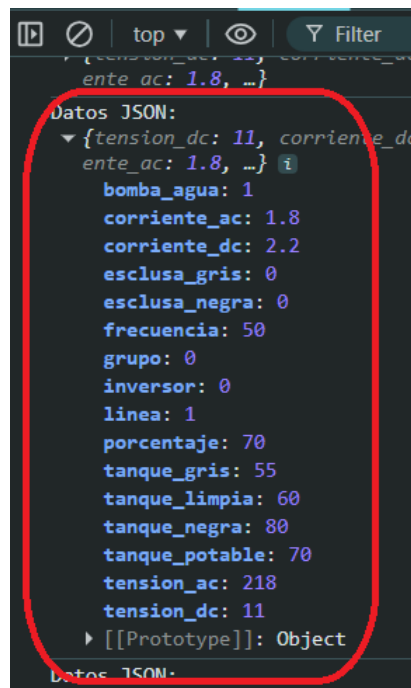


FIGURA 4.7. Archivo JSON recibido en la pestaña home.

Por otra parte, se evaluó el envío de comandos desde la aplicación hacia el ESP32 mediante solicitudes POST. La figura 4.8 muestra el mensaje recibido por el microcontrolador al accionar el pulsador correspondiente a la bomba de agua.

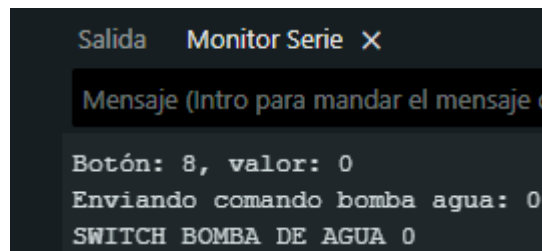


FIGURA 4.8. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.

En este caso, el identificador del comando y el valor asociado fueron correctamente interpretados por el sistema, lo que dio lugar al envío de la instrucción correspondiente hacia el módulo PRC10.

Asimismo, se verificó la respuesta del servidor ante estas solicitudes. La figura 4.9 presenta el contenido de la respuesta enviada por el ESP32, donde se confirma el comando ejecutado junto con el nuevo estado asignado.

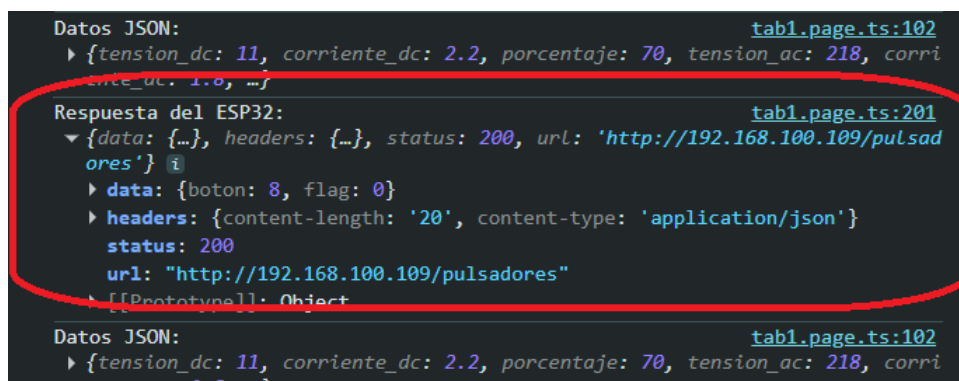


FIGURA 4.9. Respuesta del servidor ante una solicitud de control.

Como resultado, se comprobó que los datos intercambiados entre la aplicación y el ESP32 coincidían de manera consistente, que los comandos enviados eran correctamente procesados y que las respuestas del servidor reflejaban el estado actualizado del sistema.

A partir de estos ensayos, se concluyó que la comunicación entre la aplicación y el microcontrolador resultó confiable, lo que permitió validar el funcionamiento del servidor y avanzar hacia la etapa de pruebas con equipos reales.

4.4. Ensayos de laboratorio

En esta etapa se realizaron ensayos con equipos reales de la red de domótica de Enertik, con el objetivo de validar el funcionamiento del sistema en condiciones representativas de operación.

4.4.1. Prototipo experimental

Para realizar los primeros ensayos, se decidió realizar un primer prototipo del equipo. Para ello, se utilizó como base un PCB experimental perforado de tipo experimental, sobre el que se soldaron diversos conectores, cableados y componentes de forma manual.

La figura 4.10 muestra el montaje del prototipo utilizado para los ensayos iniciales.

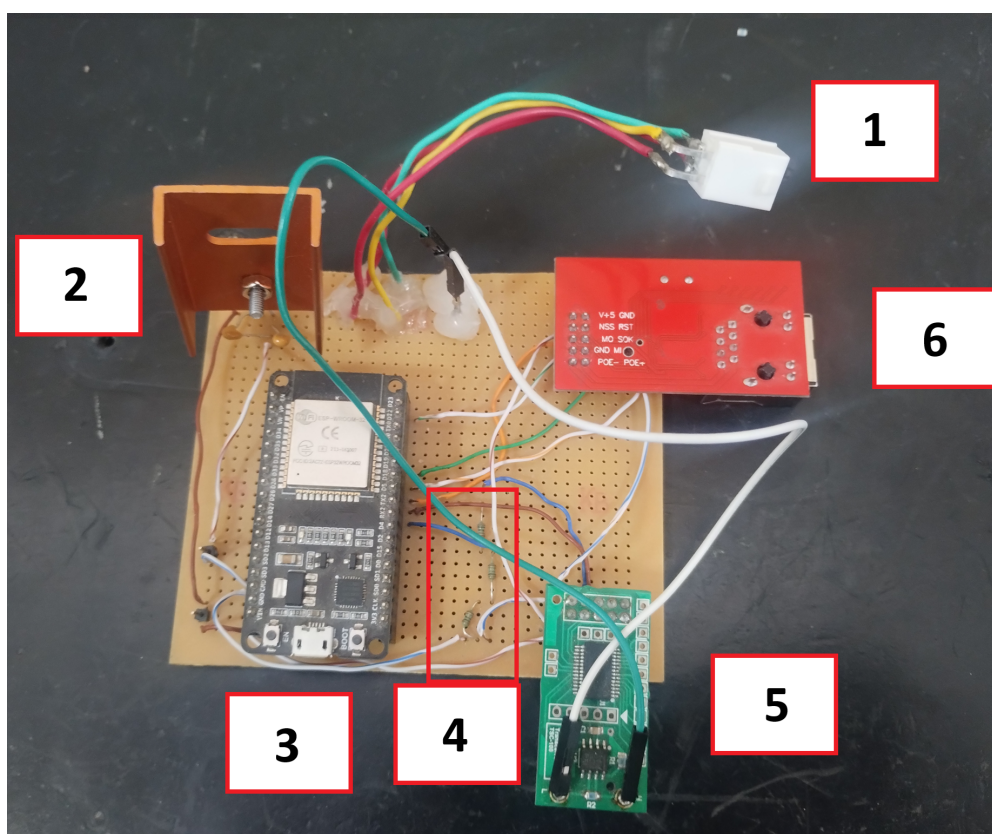


FIGURA 4.10. Prototipo experimental del equipo.

Como se puede apreciar, el prototipo cuenta con cada uno de los módulos explicados en la sección 3.1, donde:

- Componente 1: conector de cuatro hilos. Este conector es el estándar empleado en la red de domótica de Enertik. A través de él se energizan todos los componentes del PCB experimental y se accede a la comunicación CAN de la red.
- Componente 2: regulador de tensión 7805. Este dispositivo se encarga de suministrar una tensión estable de 5 V. Dado que la tensión proveniente del motorhome es de 12 V, fue necesaria su utilización para adecuar los niveles de alimentación a los requeridos por los módulos.
- Componente 3: módulo ESP32. Es el microcontrolador principal de la placa y contiene el firmware de control del sistema. En este prototipo se alimentó a través del regulador 7805, ya que la placa de desarrollo dispone internamente de reguladores que reducen la tensión a 3,3 V. Para el desarrollo final fue necesaria la incorporación de un regulador adicional dedicado al ESP32.

- Componente 4: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceptor.
- Componente 5: módulo transceptor. Se trata de una placa comercial provista por Enertik. En este prototipo se decidió incorporar un conector en el PCB experimental para facilitar su montaje y desmontaje de forma segura.
- Componente 6: módulo W5100. De manera similar al módulo transceptor, se utilizó un conector compatible que permite su montaje y desmontaje de forma sencilla y segura.

Este prototipo permitió validar el funcionamiento general del sistema y realizar pruebas iniciales de integración entre los distintos módulos de hardware.

4.4.2. Ensayos de integración del sistema completo

La figura 4.11 muestra el módulo PRC10 utilizado durante los ensayos. Este equipo fue separado y destinado específicamente a este desarrollo.



FIGURA 4.11. Módulo PRC10 utilizado en los ensayos.

Para la realización de las pruebas, se utilizó un maletín provisto por Enertik que integra los equipos comerciales empleados en aplicaciones de motorhomes. Este conjunto reproduce la configuración real del sistema, tanto a nivel de alimentación como de comunicación.

La figura 4.12 muestra dicho entorno, donde los dispositivos se encuentran interconectados mediante un cableado de cuatro conductores. En ella se identifican el prototipo desarrollado (cuadro 1) y el módulo PRC10 (cuadro 2).

El sistema dispone de dos entradas de alimentación: una fuente de 12 V en corriente continua, que representa la batería, y una entrada de corriente alterna que simula las fuentes de línea, grupo electrógeno e inversor. Como carga, se utilizaron luces testigo que permiten visualizar el accionamiento de los distintos circuitos.



FIGURA 4.12. Equipos Enertik utilizados en los ensayos.

Para la integración con el ecosistema Victron, se emplearon un módulo Cerbo GX y un equipo MultiPlus-II 12/3000/120-32 230V [37], tal como se muestra en la figura 4.13.

El procedimiento de ensayo consistió en instalar la aplicación en un dispositivo móvil con sistema operativo Android, establecer la conexión a la red Wi-Fi generada por el ESP32 y evaluar el comportamiento de cada una de las pestañas.

En la pestaña principal se accionaron los pulsadores y se verificó la respuesta de las luces testigo asociadas. Asimismo, se aplicaron variaciones sobre los sensores de nivel de los tanques, con el fin de simular distintos estados y comprobar la actualización de los indicadores en la interfaz.

Este procedimiento se replicó en las demás pestañas, donde se evaluaron los distintos controles implementados y su efecto sobre los dispositivos físicos del sistema.

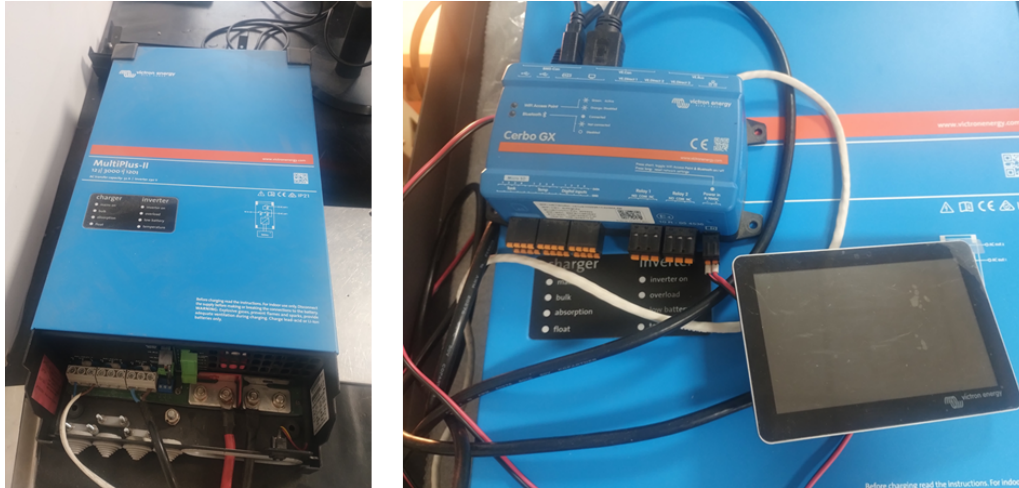


FIGURA 4.13. Equipo Victron MultiPlus-II y módulo Cerbo GX.

En relación con la comunicación con los equipos Victron, se utilizó la pestaña energía para controlar el equipo MultiPlus.

Al activar el Multiplus en modo inversor, el equipo comenzó a convertir la tensión de batería en una señal alterna de 220 V. Como carga, se conectó un soldador de tipo *cautín* de alta potencia, lo que permitió observar la variación de la potencia indicada en la aplicación.

Por otro lado, se conectó el Multiplus a la red domiciliaria y se lo configuró en modo cargador. Mediante el uso de una pinza amperométrica, se verificó que el valor de corriente de carga mostrado en la aplicación resultaba coherente con la medición real.

Como criterio de validación, se consideró satisfactorio que los estados de los actuadores coincidieran con las acciones realizadas desde la aplicación, y que las variables medidas presentaran valores coherentes con las lecturas obtenidas mediante instrumentos externos.

Los resultados obtenidos indicaron que el sistema respondió correctamente a las acciones de control, tanto en los dispositivos de la red de Enertik como en los equipos del ecosistema Victron. Las magnitudes eléctricas visualizadas en la aplicación resultaron consistentes con las mediciones realizadas.

Adicionalmente, se realizó una prueba en un motorhome real durante una visita a la fábrica, donde se evaluaron los controles en condiciones reales de operación. En esta instancia, el sistema presentó un comportamiento adecuado y consistente con los ensayos de laboratorio.

No obstante, no fue posible validar la comunicación con equipos Victron en el vehículo, debido a la ausencia de este equipamiento en la unidad ensayada. Esta verificación queda planteada como trabajo futuro.

Finalmente, debido a restricciones de la empresa fabricante, no se dispuso de registros fotográficos de los ensayos realizados en el motorhome.

Capítulo 5

Conclusiones

En este capítulo se presentan las conclusiones obtenidas de la ejecución de este trabajo y, además, se proponen perspectivas que podrían ser implementadas a futuro.

5.1. Conclusiones generales

En primer lugar, se destaca que fue posible desarrollar con éxito un dispositivo capaz de comunicarse tanto con la red de domótica de Enertik como con el ecosistema Victron.

Este resultado resulta de gran relevancia, ya que habilita la futura integración de equipos Victron dentro de la red de domótica del motorhome, amplía las capacidades del sistema y permite una mayor flexibilidad en su implementación.

Asimismo, se logró el desarrollo de una aplicación funcional que permite interactuar con el sistema de domótica. Esta aplicación es capaz de replicar la mayoría de las funcionalidades disponibles en la pantalla HMI del motorhome y brinda a los usuarios una alternativa adicional para el control del sistema.

A continuación, se destacan los principales aspectos del trabajo:

- Se desarrolló con éxito un dispositivo capaz de comunicarse con ambos ecosistemas de manera confiable.
- Se diseñó un PCB de tamaño reducido, orientado a su futura implementación como producto comercial.
- El equipo se integra a la red del motorhome mediante el cableado estándar de cuatro conductores, lo que facilita su instalación.
- Se implementó una arquitectura de firmware basada en múltiples tareas, lo que permite un funcionamiento organizado y eficiente del sistema.
- La aplicación desarrollada funciona correctamente en dispositivos Android, lo que amplía las posibilidades de uso por parte de los usuarios.
- La aplicación replica la mayoría de las funcionalidades de la pantalla HMI, lo que reduce la dependencia de esta para el control del sistema.
- Se validó el funcionamiento del sistema tanto en entorno de laboratorio como en condiciones reales de operación.
- La solución presenta un diseño escalable, lo que permite la incorporación de nuevas funcionalidades y dispositivos en el futuro.

- Se emplearon protocolos de comunicación estándar, lo que favorece la interoperabilidad con otros sistemas.

5.2. Próximos pasos

Como líneas de trabajo a futuro, se consideran diversas mejoras y extensiones del sistema desarrollado.

En primer lugar, durante los ensayos en vehículos reales, los fabricantes de motorhomes señalaron que la aplicación requiere la conexión a la red Wi-Fi generada por el ESP32, lo que implica que el teléfono celular pierde el acceso a Internet mientras se utiliza el sistema. Si bien esta situación no resulta crítica, se analizaron distintas alternativas para su resolución.

Una posible solución consiste en modificar el firmware del ESP32 para que se conecte a una red Wi-Fi existente. De esta manera, la aplicación, al conectarse a la misma red, podría acceder a los endpoints del servidor mediante la dirección IP asignada por el módem.

Otra alternativa consiste en migrar el protocolo de comunicación entre la aplicación y el ESP32. En este sentido, se podría implementar una comunicación mediante Bluetooth en reemplazo de la conexión Wi-Fi. Ambas opciones permitirían mantener la conectividad a Internet en el dispositivo móvil y, al mismo tiempo, conservar el control del sistema sin inconvenientes.

Por otra parte, se considera necesaria la adaptación de la aplicación para su ejecución en dispositivos iOS. Esta tarea no se incluyó dentro del alcance del presente trabajo.

En la actualidad, los sistemas operativos predominantes en dispositivos móviles son Android e iOS. La compatibilidad con ambos sistemas permitiría ampliar el alcance de la solución, al cubrir una mayor cantidad de usuarios potenciales.

Relacionado con este aspecto, también resulta necesario realizar modificaciones que permitan distribuir la aplicación a través de tiendas oficiales, como Play Store en Android y App Store en iOS.

En la actualidad, la aplicación solo puede instalarse mediante un archivo de tipo APK. Sin embargo, para su implementación a nivel comercial, resulta conveniente que los usuarios puedan acceder a ella a través de estas plataformas, lo que facilitaría su distribución, instalación y actualización.

Bibliografía

- [1] Enertik Argentina. Última consulta: 2025-03-14. URL: <https://enertik.com/ar/>.
- [2] Victron Energy. Última consulta: 2025-03-14. URL: <https://www.victronenergy.com.es/>.
- [3] Enertik Argentina. *PRC10: manual para el usuario*.
- [4] Espressif. Última consulta: 2025-03-21. URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html>.
- [5] Cerbo GX. Última consulta: 2025-03-16. URL: <https://www.victronenergy.com.es/communication-centres/cerbo-gx>.
- [6] *CAN Bus Explained - A Simple Intro* [2025]. Última consulta: 2025-03-16. URL: <https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial>.
- [7] *Modbus: Qué es y cómo funciona*. Última consulta: 2025-03-19. URL: <https://www.cursosaula21.com/modbus-que-es-y-como-funciona/>.
- [8] *Overview of HTTP*. Última consulta: 2025-03-19. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>.
- [9] *TCP/IP Overview*. Última consulta: 2025-03-25. URL: <https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13769-5.html>.
- [10] *Manual de GX - Modbus TCP*. Última consulta: 2025-03-15. URL: https://www.victronenergy.com/live/ccgx:modbustcp_faq.
- [11] *¿Qué es una Petición HTTP?* Última consulta: 2025-03-29. URL: <https://kinsta.com/es/blog/que-es-una-peticion-http/>.
- [12] *W5100*. Última consulta: 2025-03-19. URL: https://cdn.sparkfun.com/assets/e/2/f/c/2/W5100_Datasheet_v1_1_6.pdf.
- [13] *MCP2551*. Última consulta: 2025-03-19. URL: <https://ww1.microchip.com/downloads/en/devicedoc/20001667g.pdf>.
- [14] *Serial Communication*. Última consulta: 2025-03-29. URL: <https://learn.sparkfun.com/tutorials/serial-communication/all>.
- [15] *Serial Peripheral Interface (SPI)*. Última consulta: 2025-03-29. URL: <https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all>.
- [16] *I2C*. Última consulta: 2025-04-08. URL: <https://learn.sparkfun.com/tutorials/i2c>.
- [17] *Pulse Width Modulation (PWM): How it Works and Why it's Essential in Electronics*. Última consulta: 2025-04-02. URL: <https://www.sameskydevices.com/blog/pulse-width-modulation-pwm-how-it-works-and-why-its-essential-in-electronics>.
- [18] *PIC18F66K80 FAMILY*. Última consulta: 2025-03-19. URL: <https://ww1.microchip.com/downloads/en/DeviceDoc/PIC18F66K80%20FAMILY%20Enhanced%20Flash%20MCU%20with%20ECAN%20XLP%20Technology%2030009977G.pdf>.
- [19] *Arduino*. Última consulta: 2025-03-25. URL: <https://docs.arduino.cc/software/ide-v1/tutorials/PortableIDE/>.

- [20] *Github: modbus-esp8266*. Última consulta: 2025-03-25. URL: <https://github.com/emelianov/modbus-esp8266>.
- [21] *FreeRTOS*. Última consulta: 2025-03-19. URL: <https://www.freertos.org/>.
- [22] *Introduction to Ionic*. Última consulta: 2025-03-19. URL: <https://ionicframework.com/docs>.
- [23] *Angular*. Última consulta: 2025-03-25. URL: <https://angular.dev/>.
- [24] *What is TypeScript?* Última consulta: 2025-03-29. URL: <https://www.typescriptlang.org/>.
- [25] *HTML: HyperText Markup Language*. Última consulta: 2025-03-29. URL: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [26] *CSS: Cascading Style Sheets*. Última consulta: 2025-03-29. URL: <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [27] *Capacitor web*. Última consulta: 2025-03-25. URL: <https://capacitorjs.com/>.
- [28] *Android Studio*. Última consulta: 2025-03-25. URL: <https://developer.android.com/studio?hl=es-419>.
- [29] *¿Qué es un conector RJ45? Todo lo que necesitas saber*. Última consulta: 2025-04-15. URL: <https://seetronic.com/es/blog/what-is-an-rj45-connector/>.
- [30] *Altium Designer*. Última consulta: 2025-03-29. URL: <https://www.altium.com/es/altium-designer>.
- [31] *AP63200/AP63201/AP63203/AP63205*. Última consulta: 2025-03-29. URL: <https://4donline.ihs.com/images/VipMasterIC/IC/DIOD/DIOD-S-A0007089856/DIOD-S-A0007089856-1.pdf?hkey=CECEF36DEECD6468708AAF2E19C0C6>.
- [32] *LM340, LM340A and LM7805 Family Wide VIN 1.5-A Fixed Voltage Regulators*. Última consulta: 2025-03-29. URL: <https://www.ti.com/lit/ds/symlink/lm340.pdf>.
- [33] *What is a gerber file and how are they used in the PCB fabrication process*. Última consulta: 2025-03-29. URL: <https://resources.altium.com/es/p/what-gerber-file-pcb-fabrication-process>.
- [34] *What is RingBuffer?* Última consulta: 2025-03-30. URL: <https://medium.com/@kolheankita15/what-is-ringbuffer-7b6b808f33e0>.
- [35] *Introducing JSON*. Última consulta: 2025-04-02. URL: <https://www.json.org/json-en.html>.
- [36] *Android Debug Bridge (adb)*. Última consulta: 2025-04-02. URL: <https://developer.android.com/tools/adb?hl=es-419>.
- [37] *Victron MultiPlus-II 12/3000/120-32 230V*. Última consulta: 2025-04-02. URL: <https://enertik.com/ar/tienda/inversores/inversores-cargadores/sin-regulador-solar/victron-multiplus-ii-123000120-32-230v/>.